

Platform Incident Response Runbook

1. Purpose and Scope

This runbook covers first-response procedures for the platform team's five most common production incidents. It is written for an on-call engineer who has basic familiarity with Kubernetes, our CI/CD pipeline, and our core services, but who may not have touched the specific failing component before. Each section below follows the same shape: what the symptom looks like, why it typically happens, and the exact steps to take first. Escalate to the secondary on-call if a section's steps do not resolve the issue within twenty minutes.

2. On-Call Escalation Policy

The primary on-call engineer is paged first for any severity-1 or severity-2 incident. If there is no acknowledgment within five minutes, the page automatically escalates to the secondary on-call, and then to the engineering manager after a further five minutes. Severity-1 incidents additionally open an incident channel and require a written status update every fifteen minutes until resolution. Do not wait for permission to escalate early if you are unsure of the blast radius of an issue -- escalating a false alarm costs a few minutes; failing to escalate a real outage costs much more.

3. Kubernetes Pod Troubleshooting: OOM Kills and Crash Loops

A pod that has been killed by the out-of-memory killer shows status `OOMKilled` when you run `kubectl describe pod` on it, and its exit code will be 137. This means the container tried to use more memory than its configured limit allowed, and the kernel terminated it to protect the node. The first fix is almost always to raise the memory limit in the deployment manifest, but before doing that, check whether memory usage is growing steadily over time in the dashboards -- that pattern points to a memory leak in the application rather than an under-provisioned limit, and raising the limit will only delay the same crash. A pod stuck in `CrashLoopBackOff`, by contrast, usually means the container's own startup process is exiting immediately, often because of a missing environment variable, a bad configuration file, or an unhandled exception at boot. Always read the last fifty lines of the pod's logs before restarting it blindly; restarting without reading the logs just repeats the same crash loop.

4. Database Connection Pool Exhaustion

When the application logs show errors like connection pool exhausted or timeout waiting for connection, the database connection pool has run out of available connections, usually because a small number of long-running queries are holding connections open far longer than expected. Start by querying `pg_stat_activity` (for Postgres) or the equivalent for your database engine to find any query that has been running for more than sixty seconds, and consider terminating it if it is not a scheduled batch job. A recurring version of this problem is usually fixed by lowering the statement timeout so a single runaway query cannot hold a connection indefinitely, and by adding an alert on pool utilization crossing eighty percent so the team catches it before connections are fully exhausted.

5. TLS Certificate Expiry

If every HTTPS request to a service starts failing at once with a handshake error, check the certificate expiry date first -- an expired TLS certificate is one of the few failure modes that affects one hundred percent of traffic instantly, with zero warning if no expiry alert was configured. Renew the certificate through the certificate authority or, if the service already uses Let's Encrypt, verify that the automatic renewal job actually ran and did not silently fail. Every public-facing certificate in this organization should have a thirty-day expiry alert configured; if a certificate that expired did not have one, adding that alert is a required action item in the incident postmortem, not an optional follow-up.

6. DNS Resolution Failures

Internal service-to-service calls that suddenly fail to resolve a hostname, while external DNS still works fine, almost always point to the cluster's internal DNS service, CoreDNS, rather than to the wider internet's DNS. This commonly follows a node being drained or replaced, which can cause the CoreDNS pods themselves to be rescheduled and briefly become unavailable. Check that the CoreDNS pods are running and healthy, and restart the CoreDNS deployment if they are not. If DNS still fails after CoreDNS is confirmed healthy, check the relevant ConfigMap for a corrupted or accidentally edited configuration, since a bad ConfigMap can leave the pods running but still answering queries incorrectly.

7. Autoscaling and Resource Thresholds

The horizontal pod autoscaler will not scale a deployment up if the metrics server is unavailable, since it has no CPU or memory data to make a scaling decision from -- this can silently leave a service under-provisioned during exactly the traffic spike that should have triggered scaling. Verify the metrics server is reporting data with `kubectl top pods` before assuming the autoscaler configuration itself is wrong. A resource threshold that is set too close to typical usage, rather than with meaningful headroom, causes the opposite problem: constant, unnecessary scaling events that create noise without protecting against a real spike. A reasonable starting point is targeting seventy percent average utilization, with a cooldown period long enough to avoid flapping.

8. CI/CD Pipeline Rollback Procedure

If error rates spike immediately after a deployment, the fastest safe action is almost always to roll back to the last known-good release rather than trying to forward-fix the new code under incident pressure. Our pipeline supports an automatic rollback when the canary stage detects an error-rate threshold breach within the first five minutes of a rollout, but a manual rollback command is also available and should be used immediately if the automatic canary check did not catch the regression for any reason. After any rollback, the responsible engineer must open a follow-up ticket to fix the underlying bug before the next deployment attempt -- rolling back resolves the incident, it does not resolve the bug.

9. Incident Postmortem Template

Every severity-1 incident requires a written postmortem within two business days of resolution. The postmortem must include a timeline of detection and response, the root cause, the customer-facing impact, and a list of concrete follow-up actions with named owners and due dates. Postmortems in this organization are blameless by policy: the goal is identifying what allowed the failure to happen and how to prevent a repeat, never assigning individual fault. A postmortem without at least one concrete, assigned follow-up action is considered incomplete, regardless of how thorough the root-cause analysis section is.

10. Alerting and On-Call Fatigue

An alert that fires more than a few times a week without requiring any action from the on-call engineer should be treated as a bug in the alerting configuration, not tolerated as background noise. Alert fatigue from excessive low-priority paging is one of the leading causes of a real incident being missed or acknowledged too slowly, because engineers understandably start responding to pages more slowly once they have learned that most of them are not urgent. Review paging volume per alert on a monthly basis, and either raise the severity threshold, add a longer evaluation window, or delete any alert that has not led to a real action in the last three months.